# Inheritance

Prof. Ajay Bakre & Ahana Pradhan

# Inter-class Relationships in OO Design

- Provide a way of interaction between objects of different classes
- Types of inter-class relationships:
  1. Composition: the object and its member have a strong part-whole relationship; characterized by "part-of" relationship, e.g. a kitchen is part of a house
  2. Aggregation: a weaker form of composition, where the object and its member form a "has-a" relationship, e.g. library has shelves, books etc.
  3. Association: a weaker form of aggregation, where the object forms a "uses-a" / "references-a" relationship with another object, e.g. student uses a car
  4. Generalization / Specialization, where the object forms "is-a-kind-of" relationship with another object

# Composition vs. Aggregation vs. Association

```
// association
class Student {
        Course* courses;
       // ...
};

// aggregation
class Whole {
        Part* part;
       // ...
};

// composition
class Whole {
        Part part;
       // ...
};
```

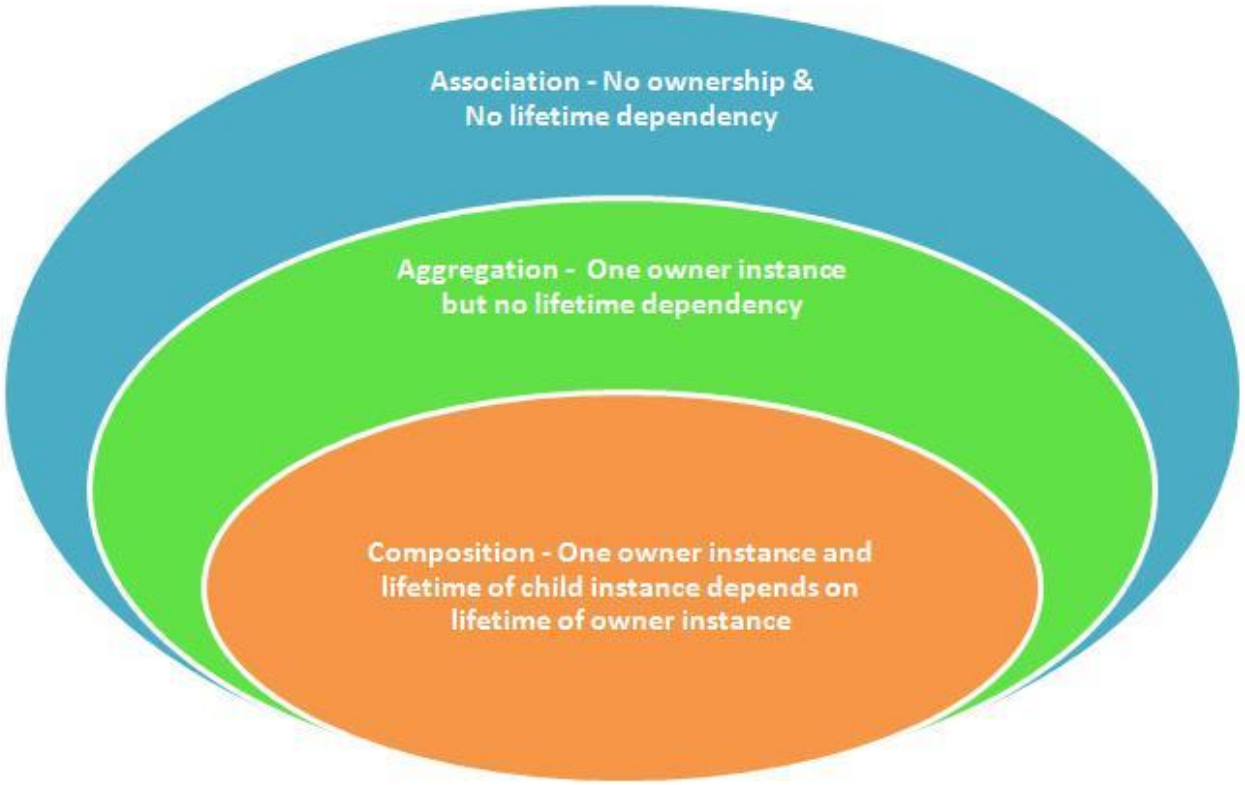



Image courtesy: https://dotnetfreakblog.wordpress.com/2014/01/11/concept-of-dependency-generalization-association-aggregation-composition-in-object-oriented-programming/

# Relationship Comparison

| Property | Composition | Aggregation | Association |
|---|---|---|---|
| Relationship type | Whole/part | Whole/part | Otherwise unrelated |
| Members can belong to multiple classes | No | Yes | Yes |
| Members' existence managed by class | Yes | No | No |
| Directionality | Unidirectional | Unidirectional | Unidirectional or bidirectional |
| Relationship verb | Part-of | Has-a | Uses-a |

Image courtesy: https://www.learncpp.com/cpp-tutorial/10-4-association/

# Initialization

- Composition – Individual parts of the object are created during object initialization
  - e.g., real and imaginary parts of a complex number
- Aggregation – An aggregate may be given the individual objects as parameters during initialization or it may start empty and objects are added later
  - e.g., initialize an array with elements or add elements later
- Association – A link may be established between the associated objects as part of creation of one object or it may happen sometime later
  - e.g. create an association between an order and a customer
  - or create an association between a student and a course on enrollment

# Inheritance

- Captures the inter-class relationship of Generalization / Specialization
- Inheritance in C++ allows a class being defined in terms of a pre-existing class
  - This produces a new class having all the functionality of the older class, and additionally defining its own specific functionality
  - Instead of composition, where a given class “contains” another class, we here introduce “derivation”, where a given class is-a(-type-of) or is-implemented-in-terms-of another class
  - The pre-existing class in an inheritance is called the **base class**, while the new class is called the **derived class**
  - A derived class “specializes” the pre-existing base class
  - A base class “generalizes” all of its derived classes

# Example

- Grandmother
  - Simple minded
  - Good English orator
  - Too religious
- Mother
  - Simple minded <no-change>
  - Good English orator <no-change>
  - Moderately religious <change>
  - Good English writing skills <new>
- Daughter
  - Simple minded <no-change>
  - Good English & Hindi orator <combination>
  - Not religious <change>
  - Plays competitive sport  <new>

# C++ Example

```
class Vehicle {
        size_t _mass;
    public:
        // constructors/destructor
        Vehicle();
        Vehicle(size_t mass): _mass(mass);
        ~Vehicle();
        // get/set functions
        size_t mass() const;
        void setMass(size_t mass);
};
```

```
class Land_Vehicle: public Vehicle {
        size_t _speed;
    public:
        // constructors/destructor
        Land_Vehicle();
        Land_Vehicle(size_t mass, size_t
                        speed) ;
        // get/set functions
        void setSpeed(size_t speed);
        size_t speed() const;
};
```

# Inheritance vs Composition

```
class Land_Vehicle: public Vehicle {
        size_t _speed;
    public:
        // constructors/destructor //
        Land_Vehicle();
        Land_Vehicle(size_t mass, size_t speed) {
                setMass(mass); // inherited member
                _speed = speed;
        }
        // get/set functions //
        void setSpeed(size_t speed) {_speed = speed;}
        size_t speed() const {return _speed);
};
```

```
class Land_comp {
    Vehicle d_v ; // composed Vehicle
    size_t _speed ;
    public:
        // constructors/destructor //
        Land_comp() ;
        Land_comp(size_t mass, size_t speed) ;
        ~Land_comp() ;
        // get/set functions //
        void setMass(size_t mass) {
            d_v.setMass(mass) ; // pass through
        }
};
```

# Access Control with Inheritance

| Access Specifier | Same Class | Derived Class | Any Other Class | Friend Function | Friend Class |
|---|---|---|---|---|---|
| Private | Yes | No | No | Yes | Yes |
| Protected | Yes | Yes | No | Yes | Yes |
| Public | Yes | Yes | Yes | Yes | Yes |

Image Courtesy: http://www.startertutorials.com/blog/wp-content/uploads/2016/09/access-specifiers-in-cpp.png

# Access Control Example

```
class Index {
    private :
        int count ;
    public :
        Index( ) {
            count = 0;
        }
        void display( ) {
                cout << count ;
        }
        void operator ++ ( ) {
                count ++ ;
        }
};
```

```
class Index1  : public
Index {
    public :
        void operator --( ) {
            count-- ;
          // undeclared
variable count
        }
};
```

# Access Control Example contd

```
class Index {
    protected :
        int count ;
    public :
        Index( ) {
            count = 0;
        }
        void display( ) {
                cout << count ;
        }
        void operator ++ ( ) {
                count ++ ;
        }
};
```

```
class Index1 : public Index {
    public :
        void operator --( ) {
            count -- ; // OK now
        }
};
int main( ) {
    Index i;
    i.display( ) ; // i == 0
    ++i ; // i.operator ++( )
    i.display( ) ; // i == 1
    --i ; // no such operator
    i.display( ) ; // i == 0
}
```

# Access Control Example contd

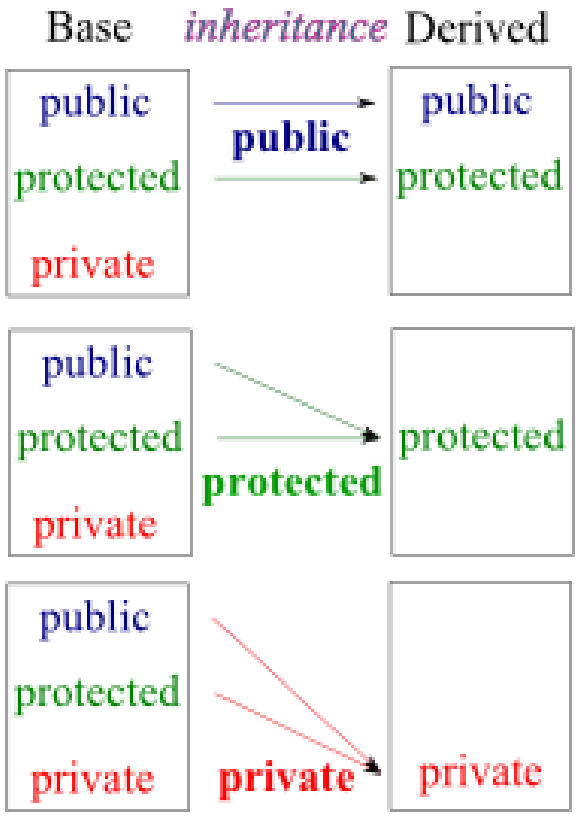
```
class Index {
    protected :
            int count ;
    public :
            Index( ) {
               count = 0;
            }
            void display( ) {
                    cout << count ;
            }
            void operator ++ ( ) {
                    count ++ ;
  }
};
```

```
class Index1 : public Index {
    public :
            void operator --( ) {
                count -- ;  // OK now
            }
};
int main( ) {
            Index1 i;
            i.display( ) ; // i == 0
     ++i ; // i.operator ++( )
     i.display( ) ; // i == 1
     --i ; // OK now
     i.display ) ; // i == 0
}
```

# Access Control and Encapsulation

- A derived class does not have access to *private* members of the base class
- The keyword *private* enables data hiding, and *public* enables communication with users
- In a well-designed class, only its objects are in full control of their data
  - Private data members with public methods to access them
- **The keyword *protected* is used for members of the base class which are not in its public interface, but can be accessed by derived classes**
- Good practice:
  - If a derived class must have access to the private data of the base class, then use member functions (accessors and modifiers) which are declared protected in the interface of the base class
- Maintain data integrity using encapsulation and data hiding
  - Encapsulation restricts access to functionality of objects
  - Data hiding restricts control over the data of an object exclusively to the members of its class

# Modes of Inheritance




- Public Inheritance
  - class Derived : public Base
    - No change in access control of Base class members in Derived class
- Protected Inheritance
  - class Derived : protected Base
    - Public and protected members of Base class become protected in Derived class
- Private Inheritance
  - class Derived : private Base
    - All members of Base class become private members in Derived class

Image courtesy: https://www.bogotobogo.com/cplusplus/multipleinheritance.php

# Summary of Inheritance in C++

| Base class member access specifier | Type of Inheritence | | |
|---|---|---|---|
| | Public | Protected | Private |
| Public | Public | Protected | Private |
| Protected | Protected | Protected | Private |
| Private | Not accessible (Hidden) | Not accessible (Hidden) | Not accessible (Hidden) |

Image courtesy: https://eng.libretexts.org/Courses/Delta_College/C_-_Data_Structures/04%3A_Inheritence/4.04%3A_Modes_of_Inheritance

# Which Mode of Inheritance to Use?

- Private inheritance is the default mode of inheritance
  - Needed in situations where a derived class object is defined “in-terms-of” the base class, but where composition cannot be used.
- Public inheritance should be used to define an is-a relationship between a derived class and a base class (most commonly used)
  - A derived class object “is-a” base class object allowing the derived class object to be used polymorphically as a base class object in code that expects it
- Protected inheritance is not widely used
  - May be useful when defining a base class that is itself a derived class and needs to make its base class members available to classes derived from itself

# Private and Protected Inheritance

```
class Engine {
public:
    void start() { cout << "Engine started\n"; }
    void stop()  { cout << "Engine stopped\n"; }
};

class Car : private Engine {   // private inheritance
public:
    void drive() {
        start();   // Car uses Engine internally
        cout << "Car is driving\n";
    }
    void park() {
        cout << "Car is parking\n";
        stop();
    }
};
```

```
class Device {
public:
    void powerOn()  { cout << "Device powered on\n"; }
    void powerOff() { cout << "Device powered off\n"; }
};

class Computer : protected Device {   // protected inheritance
public:
    void boot() {
        powerOn();
        cout << "Computer booting...\n";
    }
};

class Laptop : public Computer {
public:
    void openLid() {
        powerOn();  // ✅ accessible (protected in Computer)
        cout << "Laptop lid opened\n";
    }
};
```

# Which method gets called?

```
class Index {
    protected :
            int count ;
    public :
             Index( ) {
                        count = 0;
             }
  void display( ) {
                    cout << “Index=” << count ;
             }
  void operator ++ ( ) {
     count ++ ;
  }
};
```

```
class Index1 : public Index {
    public :
            void operator --( ) {
                    count -- ;
            }
  void display( ) {
    cout<< “Index1=” << count ;
  }

};
int main( ) {
   Index1 i;
   i.display( ); // Index=0 or Index1=0?
}
```

# Order of Constructor Calls

- Since a derived class object is constructed or built on top of the base class object
  - Base class object must be constructed before the derived class object is initialized
  - Constructor for derived class is responsible for proper initialization of data members in the base class
- Problem with calling base class constructor from inside derived class constructor
  - Since the base class object must be created before derived class constructor is called, you cannot call the base class constructor inside the derived class
  - A base class initializer can be used to initialize the base class object instead
  - If no base class initializer is supplied, compiler inserts a default one
- Only after the base class object is constructed successfully, data members of derived class are available for initialization

# Calling Methods vs Constructors (1)

```
class Sample {
  public :
       void fun1( ) {
             cout << "Hi" ;
       }
};
class NewSample : public Sample {
  public :
       void fun1( ) {
             cout << "Bye" ;
         Sample :: fun1( ) ; // normal calling
     }
};

int main() {
       NewSample s;
        s.fun1();
}
```

```
class Sample {
  public :
    Sample( ) {
          cout << ”Sample’s Constructor" ;
    }
};
class NewSample : public Sample {
  public :
    NewSample( ) {
          Sample::Sample(); // Too late
          cout << ”NewSample’s Constructor" ;
    }
};
int main() {
    NewSample s;
}
```

# Calling Methods vs Constructors (2)

```
class Sample {
  public :
    void fun1( ) {
        cout << "Hi" ;
    }
};
class NewSample : public Sample {
  public :
    void fun1( ) {
        cout << "Bye" ;
        Sample :: fun1( ) ; // normal calling
    }
};
int main() {
    NewSample s;
    s.fun1();
}
```

```
class Sample {
  public :
    Sample( ) {
        cout << ”Sample’s Constructor" ;
    }
};
class NewSample : public Sample {
  public :
    NewSample( ) : Sample() { // using initializer list
        cout << ”NewSample’s Constructor" ;
    }
};
int main() {
    NewSample s;
}
```

# Constructors with Arguments

```
class Sample {
  private :
      int i ;
  public :
      Sample ( int j ) {
            i = j;
      }
} ;
class NewSample : public Sample {
  public :
      NewSample ( int k ) : Sample ( k ) {
      }
} ;
```

```
int main () {
    NewSample s(15);
}
```

If base class initializer is not explicitly called, compiler will insert a 0-arg constructor by default

# Constructors with Arguments

```
class Sample {
  private :
      int i ;
  public :
        Sample ( int j ) {
                i = j;
        }
} ;
class NewSample : public Sample {
  public :
        NewSample ( int k ) : Sample ( k )
{
      }
} ;
```

```
int main () {
        NewSample s(15);
}
```

You cannot provide anything other than an expression in the base initializer so ”Sample(k < 0? 0 : k)” is fine but can’t put if statements

# Why Base to Derived Initialization?

```
class Sample {
  protected : int i ;
  public :
      Sample ( int j ) {
            i = j;
      }
} ;
class NewSample : public Sample {
  private : int m;
  public :
      NewSample ( int k ) : Sample ( k ) {
            m = 2 * i ;
      }
} ;
```

```
int main () {
    NewSample s(15);
}
```



Derived class initialization can depend on base class data members to initialize its own data members

# Initializing Data Members

```
class Vehicle {
  private :
     int _mass;
  public :
     Vehicle ( int mass ) {
          _mass = mass;
     }
} ;
class LandVehicle : public Vehicle {
  private :
     int _speed;
  public :
     LandVehicle ( int mass, int speed ) :
          Vehicle (mass), _speed (speed) {
     }
} ;
```

```
int main () {
    LandVehicle l_v (15, 20);
}
```



Initializer for base class object and data member of derived class object on a single line. Base class object must be initialized before data members of derived class obj.

# Order of Destructor Calls

- When an object is destroyed or goes out of scope, destructors are automatically called
- When an object of a derived class is destroyed or goes out of scope
  - The destructor of derived class is called first
  - The destructor of base class is called afterwards
  - Order of destruction is the opposite of order of construction

# Rules for Constructors and Destructors

- If ANY constructor is defined for a class, the compiler suppresses generating default constructor
- If ANY of the following is defined for a class, the compiler suppresses generating the default for ALL of these
  - Copy operation
  - Move operation
  - Destructor
- Note that some non-ISO implementations may still generate copy constructor and assignment operator if a destructor is defined but this feature is deprecated in ISO releases (C++98 and later)
- Default constructors / destructors can be reinstated or deleted explicitly
- Base class object must be explicitly copied / moved in a derived class copy / move constructor or operator, if defined

# Constructors Examples

```
/* default and delete */
class Sample {
  protected : int i ;
  public :
     Sample ( int j ) {
          i = j;
     }
     Sample () = default; /* default
        constructor is re-instated */
     Sample (const Sample& s) =
          delete; /* default copy constructor
                     is explicitly deleted */
     Sample& operator=(const Sample& s) =
          delete; /* default assignment
               operator is explicitly deleted */
} ;
```

https://github.com/ahanapradhan/OOPs/blob/main/call_cons_copy.cpp

# Nested Inheritance

- Inheritance corresponds to a tree data structure
- Multi-level tree gives rise to nested inheritance
  - e.g. Truck derived from Land_Vehicle, which is derived from Vehicle
- However, nested inheritance must be used with caution
  - Repeatedly deriving classes from classes quickly results in big, complex class hierarchies that are hard to understand, hard to use and hard to maintain
  - Data hiding is hard to implement in complex class hierarchies

```
class Vehicle {
private:
    string manufacturer;   // company name, private
    string model;          // model name, private
protected:
    string status;         // protected
public:
    Vehicle(const string& manuf, const string& mdl)
        : manufacturer(manuf), model(mdl), status("active") {}

    string getManufacturer() const { return manufacturer; }
    string getModel() const { return model; }
    string getStatus() const { return status; }
};
class Land_Vehicle : public Vehicle {
protected:
    int wheel_count;      // protected
public:
    Land_Vehicle(const string& manuf, const string& mdl, int w)
        : Vehicle(manuf, mdl), wheel_count(w) {}
    int getWheelCount() const { return wheel_count; }
};

class Truck : public Land_Vehicle {
private:
    double load_capacity; // only for Truck
public:
    Truck(const string& manuf, const string& mdl, int w, double cap)
        : Land_Vehicle(manuf, mdl, w), load_capacity(cap) {}
    double getCapacity() const { return load_capacity; }
};
```

# Multiple Inheritance

```cpp
class Mammal {
public:
    void show() {
        cout << "Mammals can give direct birth." << endl;
    }
};

class WingedAnimal {
public:
    void show() {
        cout << "Winged animals can flap." << endl;
    }
};
```

```cpp
class Bat : public Mammal, public WingedAnimal {
    // Bat is a Mammal and a WingedAnimal
public:
    void show() {
        Mammal::show();
        WingedAnimal::show();
    }
};

int main() {
    Bat b;
    b.show();
    return 0;
}
```

# Multiple Inheritance

- In multiple inheritance a class is derived from several base classes
- Using multiple inheritance, the newly derived class would be considered as an instantiation of all base classes
  - If the derived class does not have a strong is-a relationship with all the base classes, composition is more appropriate
- In general, single inheritance is more widely used than the multiple one
- Good class design dictates that a class should have a single, well described responsibility (S of **SOLID principes** A Solid Guide to SOLID Principles | Baeldung).
- The *Single responsibility principle often conflicts with multiple inheritance* where we can state that objects of class Derived are both Base1 and Base2 objects

# How about this?

```cpp
// Base class for car features
class Car {
public:
    void drive() {
        cout << "Car is driving." << endl;
    }
};

// Base class for GPS features
class GPS {
public:
    void navigate() {
        cout << "GPS is navigating." << endl;
    }
};
```

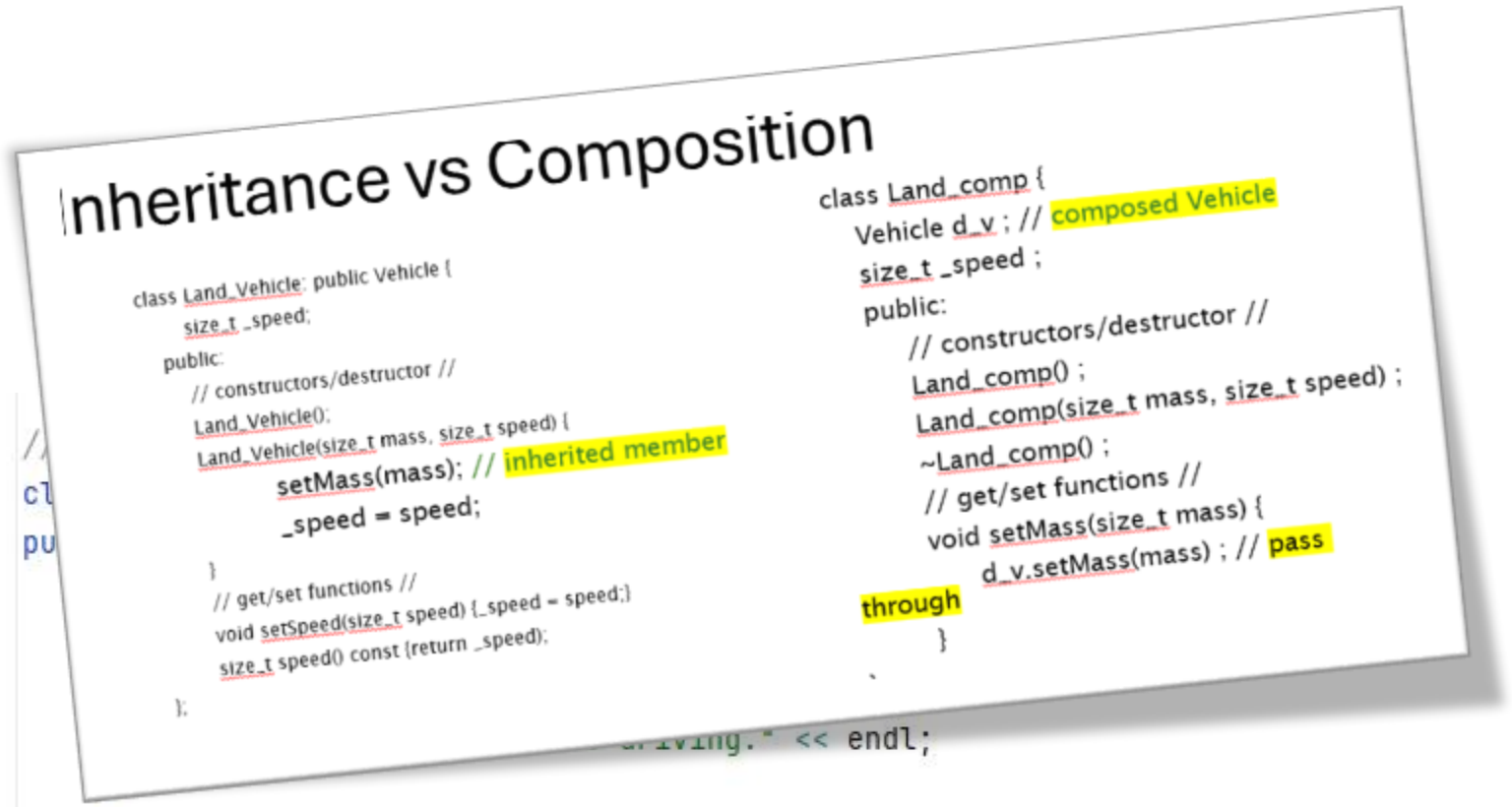
```cpp
// Derived class inherits from both Car and GPS
class SmartCar : public Car, public GPS {
public:
    void autoDrive() {
        GPS::navigate();
        Car::drive();
        cout << "SmartCar is auto-driving." << endl;
    }
};

int main() {
    SmartCar myCar;
    myCar.autoDrive();
    return 0;
}
```

# How about this?

```cpp
// Base class for car features
class Car {
public:
    void drive() {
        cout << "Car is driving." << endl;
    }
};

// Base class for GPS features
class GPS {
public:
    void navigate() {
        cout << "GPS is navigating." << endl;
    }
};
```




```cpp
};

int main() {
    SmartCar myCar;
    myCar.autoDrive();
    return 0;
}
```

# Real Life  Design Challenge

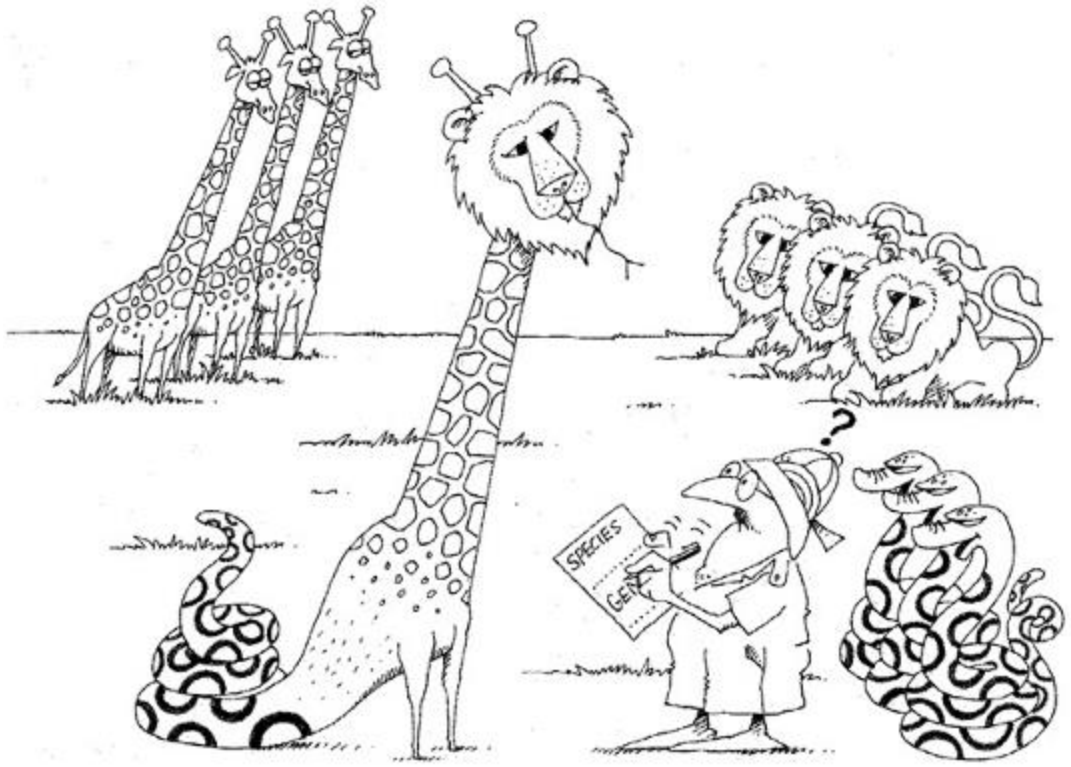
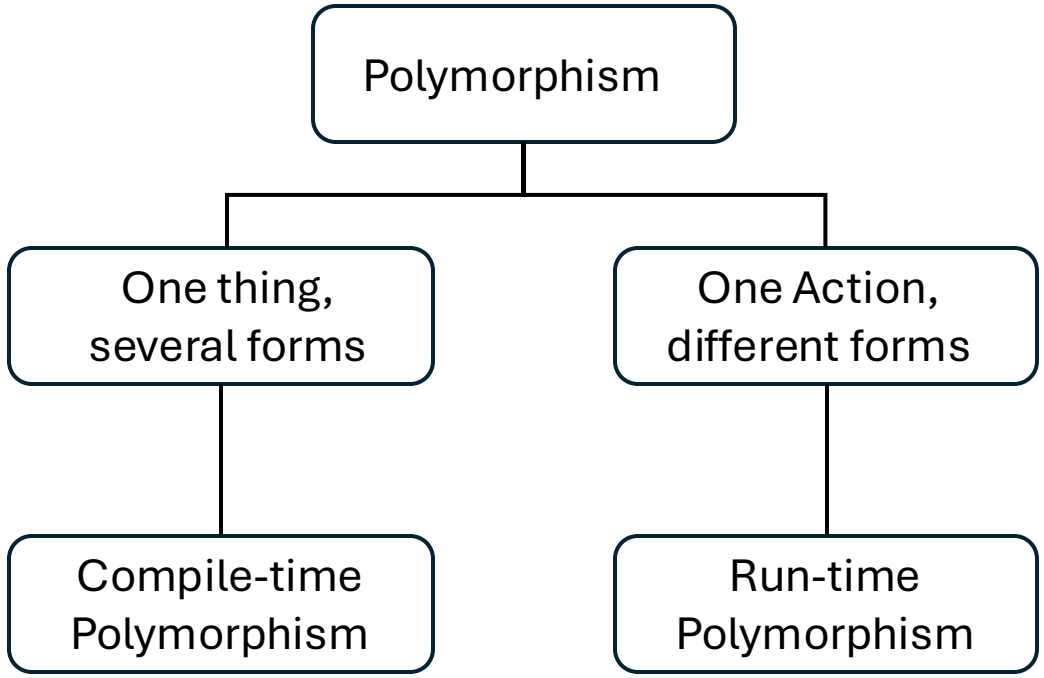



Image source: Object-Oriented Analysis and Design with Applications by Grady Booch et al.

# Polymorphism

# Polymorphism



# Compile-time Polymorphism

- One action, several forms:
  - Function overloading, e.g.
  - Operator overloading
- Binding – deciding which function to call
- Compile-time polymorphism – binding is done during compilation

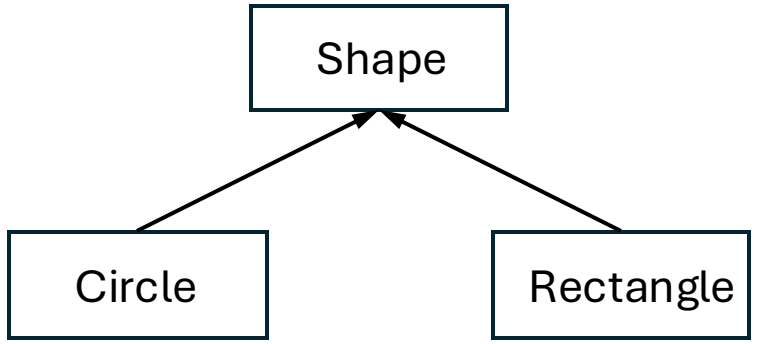
```
i = abs ( -24 ) ;
a = abs ( 1.5f ) ;
l = abs ( -12L ) ;
```

```
int i, j, k ;
k = i + j;
Comp c1, c2, c3 ;
c3 = c1 + c2 ;
```

# Run-time Polymorphism

- One action, different activities, e.g.
  - Mouse click → Close window
  - Mouse click → Display Menu
- Action – function call
- Activity - Execute different functions at different times
- Run time Polymorphism - Binding is done at run-time

# Problem Statement




- Requirements
  - *App should be able to draw Rectangles & Circles*
  - *Order of drawing should be preserved*
- Solution
  - *Inheritance, Upcasting, Virtual Function*

# Upcasting – Derived to Base

```
5  class Shape {
6  public :
7      Shape() = default;
8      void draw() {
9          cout << "Shape";
10     }
11 };
12
13 class Circle : public Shape {
14 public :
15     Circle() = default;
16     void draw() {
17         cout << "Circle";
18     }
19 };
20
21 class Rectangle : public Shape {
22 public :
23     Rectangle() = default;
24     void draw() {
25         cout << "Rectangle";
26     }
27 };
```

```
int main() {
    Shape *p;
    Circle c;
    Rectangle r;
    p = &c; // Upcasting,
    p->draw(); //
    p = &r; // Upcasting
    p->draw();
}
```

# Object Slicing

```
class Shape {
    int id;
protected:
    int getid() {
        return id;
    }
public :
    Shape(int id) {this->id = id;}
    Shape(const Shape&) = default;
    void draw() {
        cout << "["<< this << "] Shape[" << getid() << "]" << endl;
    }
};

class Circle : public Shape {
public :
    Circle(int id): Shape(id) {}
    Circle(const Circle&) = default;
    void draw() {
        cout << "["<< this << "] Circle[" << getid() << "]"  << endl;
    }
};

class Rectangle : public Shape {
public :
    Rectangle(int id): Shape(id) {}
    Rectangle(const Rectangle&) = default;
    void draw() {
        cout << "["<< this << "] Rectangle[" << getid() << "]" << endl;
    }
};
```

```
int main() {
    Shape p{ id: 13}, q{ id: 2};
    p.draw();
    q.draw();
    Circle c{ id: 0};
    Rectangle r{ id: 12};
    c.draw();
    r.draw();

    p = c;
    p.draw();
    q = r;
    q.draw();
    c.draw();
    r.draw();

}
```

```
C:\Users\ahana\Desktop\solution\mai
[0xcd2e7ff7bc] Shape[13]
[0xcd2e7ff7b8] Shape[2]
[0xcd2e7ff7b4] Circle[0]
[0xcd2e7ff7b0] Rectangle[12]
[0xcd2e7ff7bc] Shape[0]
[0xcd2e7ff7b8] Shape[12]
[0xcd2e7ff7b4] Circle[0]
[0xcd2e7ff7b0] Rectangle[12]

Process finished with exit code 0
```





Images: generated by ChatGPT

# Virtual Functions

```
class Shape {
public :
    Shape() = default;
    virtual void draw() {
        cout << "Shape";
    }
};

class Circle : public Shape {
public :
    Circle() = default;
    void draw() {
        cout << "Circle";
    }
};

class Rectangle : public Shape {
public :
    Rectangle() = default;
    void draw() {
        cout << "Rectangle";
    }
};
```

```
int main() {
    Shape *p;
    Circle c;
    Rectangle r;
    p = &c;
    p->draw();
    p = &r;
    p->draw();
}
```

# Pure Virtual Functions

```cpp
class Shape {
public :
    Shape() = default;
    virtual void draw() = 0; // abstract base class
};

class Circle : public Shape {
public :
    Circle() = default;
    void draw() {
        cout << "Circle";
    }
};

class Rectangle : public Shape {
public :
    Rectangle() = default;
    void draw() {
        cout << "Rectangle";
    }
};
```

```cpp
26
27  int main() {
28      Shape *p;
29      Shape shape; // compilation error
30      Circle c;
31      Rectangle r;
32      p = &c;
33      p->draw();
34      p = &r;
35      p->draw();
36  }
```

Shape

ns File 11 Project Errors Qodana New

main.cpp C:\Users\ahana\Desktop\solution 11 problems

Class Shape is abstract:... :29

# Early vs. Late Binding

- Early Binding
  - If call is made using object
- When called using object
  - No late binding even if function is virtual
- Late binding
  - There must be Inheritance
  - Function signature in Base and Derived must be same
  - In Base class function, must be virtual
  - Call must be made using pointer

# Implementation of Virtual Functions

```
class Ex {
    // empty class
};
int main( ) {
    Ex e1, e2 ;
    cout << sizeof (e1) << endl;
    cout << sizeof (e2) << endl;
}
```

```
C:\Users\ahana\Desktop\solution\main.
1
1

Process finished with exit code 0
```

```
class Ex {
    void show(){}
};
int main( ) {
    Ex e1, e2 ;
    cout << sizeof (e1) << endl;
    cout << sizeof (e2) << endl;
}
```

```
C:\Users\ahana\Desktop\solution\main.exe
1
1

Process finished with exit code 0
```

```
class Ex {
    virtual void show(){}
};
int main( ) {
    Ex e1, e2 ;
    cout << sizeof (e1) << endl;
    cout << sizeof (e2) << endl;
}
```

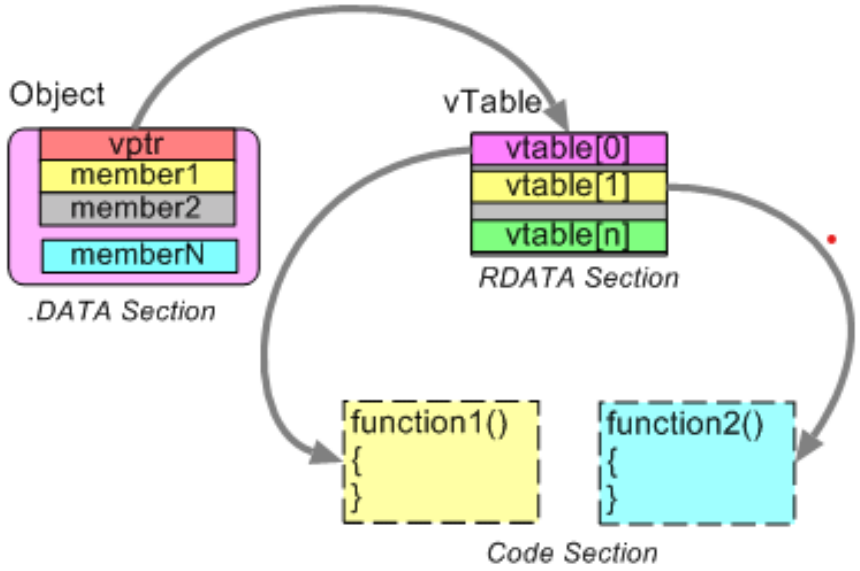
```
C:\Users\ahana\Desktop\solution\main.exe
8
8
```

```
class Ex {
    virtual void show(){}
    virtual void nop(){}
};
int main( ) {
    Ex e1, e2 ;
    cout << sizeof (e1) << endl;
    cout << sizeof (e2) << endl;
}
```

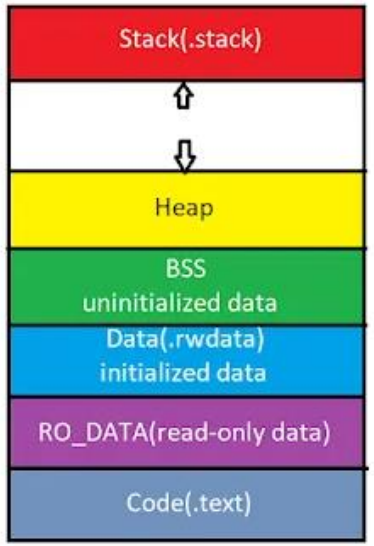


# VTABLE And VPTR

- VTABLE
  - One per class
  - Contains addresses of only Virtual functions
- VPTR
  - One per object
  - Present only if class contains a virtual function
  - VPTR always points to VTABLE

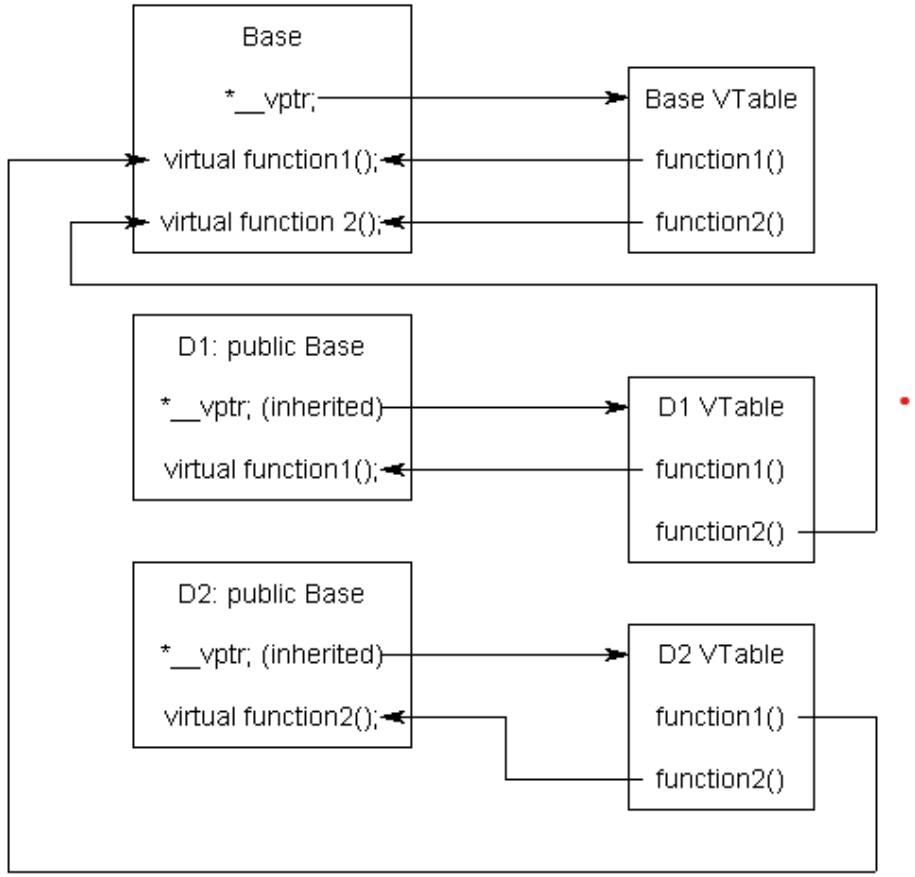



[Virtual pointer in C++ | VPTR VTABLE | VFPTR VFTABLE | Understanding]




[Program structure. Data Segment.. Data segment it is structured piece of... | by Alexey Usachov | Medium]

# VTABLE And VPTR: Inheritance




25.6 — The virtual table – Learn C++

# VTABLE / VPTR Example

```
class Shape {
public:
    virtual void draw1() {
    }

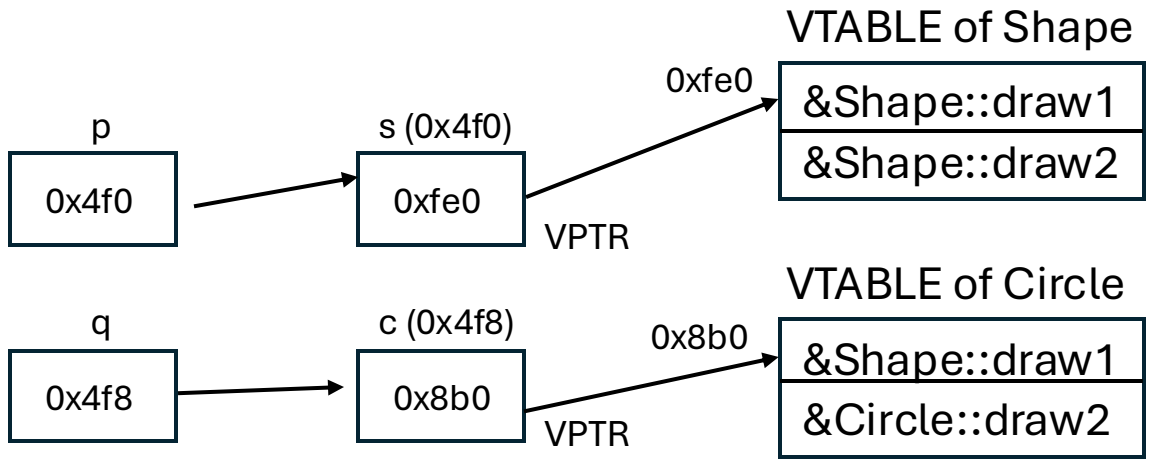
    virtual void draw2() {
    }
};
class Circle : public Shape {
public :
    void draw2() {
    }
};
```

```
int main() {
    Shape *p, *q;

    Shape s;
    Circle c;

    p = &s;
    p->draw2();

    q = &c;
    q->draw2();
}
```



# More Example

```
class Building {
public:
    virtual int GetFloorsCount() const { return FloorsCount; }
    int FloorsCount;
};

class Educational {
public:
    virtual void PassExams() {}
};

class School : public Building, public Educational {
public:
    virtual void OpenLibrary() {}
    int TeachersCount;
};

int main() {
    School s;
    Building& b = s;
    Educational& e = s;

    cout << "Building    - " << &b << " size - " << sizeof(b) << endl;
    cout << "Educational - " << &e << " size - " << sizeof(e) << endl;
    cout << "School      - " << &s << " size - " << sizeof(s) << endl;
```




```
Building    - 0x33425ff790 size - 16
Educational - 0x33425ff7a0 size - 8
School      - 0x33425ff790 size - 32

Process finished with exit code 0
```

[Multiple Inheritance for a Polymorphic Class in C++ | by Alexey Usachov | Medium]

# Using Upcasting and Virtual Functions

- Upcasting
  - Exploits inheritance (is-a relationship)
  - Only works when inheritance mode is “public”
  - Allows common data structures and code to manipulate objects of base and derived classes, e.g. arrays of pointers, lists
- Virtual Functions
  - Allow class-specific code to be executed via dynamic binding, e.g. circle and rectangle need different draw functions
  - In addition, pure virtual functions allow abstract classes that have no objects but are only used to derive other classes

# Summary of Polymorphism

- Upcasted pointer
  - `Base *b; Derived d; b=&d;`
- Abstract class
  - Contains at least one pure virtual function
  - May contain non-virtual function
  - Object creation not possible
- Virtual function mechanism
  - Call through an Upcasted pointer
  - If function signatures are same
    - Base class version if non-virtual
    - Derived class version if virtual
- A virtual function is *early bound* (compile-time) if **called through object**
- A virtual function is *late bound* (run-time) if called **through a pointer** (upcasted or not)

# Misc Example

```
class Shape {
    float f_member;
    int f_int;
public:
    static int s_int;
    virtual void test(){}
};

class Rectangle : public Shape {
    void test() {
        cout << "Hello World!" << endl;
    }
};
class RedRectangle : public Rectangle {
    void test() {
        cout << "Hello Red World!" << endl;
    }
};
```

```
int Shape::s_int = 0;
int main() {
    Shape *p;
    p = new Rectangle;
    p->test();
    cout << sizeof(*p) << endl;
    cout << ++Rectangle::s_int << endl;
    delete p;
    p = new RedRectangle;
    p->test();
    cout << sizeof(*p) << endl;
    cout << RedRectangle::s_int << endl;
    delete p;
}
```